

Share

 Comment Star

...

Tiny-ImageNet-200 classification using CNN models

Отчет по 2 домашке (соревнованию на Kaggle по классификации TinyImageNet200) по ML3. Лиза Фоменко

[lizzzafomenko](#)

Created on October 27 | Last edited on October 27

ССЫЛКА на папку на Гугл диске с solution.py, ML3_HW2.ipynb, models.py (со всеми моделями, которые я описала в отчете):

<https://drive.google.com/drive/folders/1XRyf21Kdda2mQAqrbqApGGthU0sIDqMe?usp=sharing>

Но ML3_HW2.ipynb написан под работу на сервере. в колабе была куча проблем со скатыванием датасета :(прошу прощения

Главная проблема наших данных - это их очень малое количество, так как все наши картинки обрезаны до 64x64 (а не 256x256 и др). На таких данных большинство сетей просто переучиваются.

Сначала я решила бороться с этой проблемой с помощью аугментации данных - torchvision.transforms. Я использовала повороты, отражения, добавление шума и контрастирование, а также для всех входных данных (и тестовых тоже) я использовала torchvision.transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). Это базовая нормировка в torch, основанная на ImageNet, но использовать её для всех картинок является нормальной практикой (<https://docs.pytorch.org/vision/main/transforms.html>). Я увеличила размер train_dataset в 7 раз за счет данных преобразований.

Важно, что я дополнительно разбила val выборку на две: val и test_for_tuning. val выборка использовалась в качестве валидационной, а test_for_tuning для тестовой оценки модели (см графики test_acc и test_loss). В эту выборку вошли по 2 картинки каждого из классов с номерами _0 и _1 из val выборки. Я привыкла так делать, поэтому сделал доп выборку.

В моем решении используется OmegaConf, я нахожу его оч удобным. Я пробовала Adam, AdamW, SGD в качестве оптимизаторов, ConstantLR, CosineLR и OneCycleLR в качестве scheduler. Лучшее решение было выбито на AdamW и ConstantLR с factor = 0.5

Далее я решила попробовать самые базовые модели: AlexNet, ResNet. AlexNet был очень близок к порогу на val_acc при one-shot использовании, но начал переобучаться (видим по графику val_loss). То же самое случилось и с ResNet - обе модели быстро переобучались. Также из "классики" я попробовала VGG13, но его нет на графике, так как эта модель показала себя просто ужасно (val loss взлетел практически сразу и все графики стали выглядеть неадекватно). Поэтому графики для VGG13 не показаны, но его архитектура есть в models.py. AlexNet выдал test_acc = 0.4975, а ResNet - 0.485

И в ResNet, и в AlexNet количество параметров для нашей задачи слишком огромно: 18 и 57 миллионов соответственно (у нас 100 тысяч train изображений по 12288 значений каждое). Поэтому я решила уменьшать количество параметров. Я делал это именно на AlexNet, так как эта архитектура очень базовая и для всяких добавок требует сильно меньше телодвижений, чем ResNet.

AlexNetSmallerVersion - это полностью архитектура AlexNet, но количество каналов сильно урезано. Эта версия модели уже не переобучалась (и train_loss, и val_loss падают на протяжении всего обучения), что вылилось в понижении test_acc = 0.46. Возможно, использование хороших настроек optimizer, scheduler, batch_size и тд, но я решила, что это слишком скучно.

Параллельно с этой домашкой я работала в лабе и использовала seq2act ДНК-модель для предсказания транскрипционной активности участков генома. Эта модель является полностью CNN, она использует Dilated Convolutions и с помощью них показывает неплохо качество (ссылка на модель ChromBPNet - <https://doi.org/10.1101/2024.12.25.630221>). Я решила тоже использовать такие свертки, потому что они позволяют дешево увеличить receptive field. Я сделала модельку DilatedConvNet, которая полностью состоит из сверток. Но её качество оставило желать лучшего (0.405), так как она очень сильно переобучилась (хотя количество параметров было сильно меньше, чем у AlexNetSmallerVersion).

Дальше я решила работать с AlexNetSmallerVersion и вносить в нее изменения (так как именно эта модель не переобучилась на тестовых запусках и можно было добавлять новые фишки безболезненно). Сначала я добавила BatchNorm2d (то есть обнуление некоторых каналов после сверток). Это привело к небольшому улучшению качества, но я решила оставить это как регуляризацию.

Я подумала, что мне слишком жаль бросать идею использования Dilated Convolutions. Поэтому я придумала следующую вещь: использовать DilConv вместо MaxPool. Логика за этим стояла такая: MaxPooling используем для уменьшения размера картинок (вдвое), беря max / avg и тд из группы значений в канале. Использование DilConvs может привести к такому же кратному уменьшению размеров, при этом за счет dilations мы будем обрабатывать большое receptive field на этапе уменьшения размеров. Такая операция даст дополнительные параметры, но их количество невелико (+400 тысяч параметров при размере модели в 6.5M параметров). Первый запуск такой модели AlexNetSmallerVer_BN_DilConv показан желтым и производился на батчах размера 256. Качество упало по сравнению с AlexNetSmallerVer, поэтому я решила увеличить батчсайд для более гладкой сходимости - модель под названием AlexNetSmallerVerBN_512. Удвоение размера батча дало прирост в 0.04 accuracy, модель стала такой же по качеству, как ResNet.

Но ResNet использует residual connections, а моя модель - нет. Мне стало интересно, какой прирост качества дадут мне residuals. К сожалению, ряд экспериментов не отобразился в этом отчете, потому что сервер, на котором я все ставила упал, и я проводила испытания на другом сервере (у меня local wandb, поэтому запись сервер-зависимая). Использование остатков бустануло качество моей модели, но стало приводить к более быстрому переобучению, в связи с чем я решила еще сильнее уменьшить количество параметров модели - запуски с самой последней архитектурой называются FINAL_MODEL. В ней есть residuals, dilconvs на месте alexnet-овских пулингов и BatchNorm2D. Использование матча в 512 и стандартного weight_decay = 1e-4 (во всех предыдущих запусках) дало accuracy 0.535 на тесте. Последние несколько дней я просто игралась со всеми параметрами (batch_size, weight_decay, optimizer, learning rate, dropout rate и тд). Лучшим запуском оказался: batch_size = 256, weight_decay = 1e-3, weight_decay = AdamW, learning_rate = 4e-4. test_accuracy получилось 0.575. Именно это решение я использовала для предсказания меток test выборки и получила accuracy = 0.5305

Небольшие выводы: я уверена, что модели можно улучшать бесконечно. Конечно, качество, полученное мною, далеко не идеальное. Но я увидела в принципе все темы, о которых мы говорили: переобучение и способы избежать его с помощью уменьшения параметров, аугментации данных, регуляризация с помощью DropOut (2D на свертки и 1D на полносвязных слоях), BatchNorm и weight_decay. В принципе я увидела влияние и lr_rate, и weight_decay, и batch_size на сходимость модели. Все таки вывод такой: настройка гиперпараметров - работа тонкая :) не зря придумано столько методов их подбора (как, например, OneCycleLR; Optuna как инструмент подбора гиперпараметров).

Metrics





Ссылки кода:

ResNet (architecture + ResNetBlock)

<https://www.digitalocean.com/community/tutorials/writing-resnet-from-scratch-in-pytorch>

<https://towardsdatascience.com/resnets-why-do-they-perform-better-than-classic-convnets-conceptual-analysis-6a9c82e06e53/>

DilatedConvNets - idea from <https://github.com/kundajelab/chrombpnet.git>

Литература / источники, которые были изучены до начала работы

обзор CNN : <https://theaisummer.com/cnn-architectures/>

Примеры других людей по классификации ImageNet / TinyImageNet

<https://github.com/pytorch/examples/tree/main/imagenet>

https://www.researchgate.net/figure/Proposed-structure-of-the-CNN-model-for-a-64-64-color-image-four-convolutional-layers_fig2_359464348

<https://www.pinecone.io/learn/series/image-search/imagenet/>

https://cs231n.stanford.edu/reports/2016/pdfs/401_Report.pdf

<https://github.com/sithu31296/sota-backbones.git>

ImageFolder usage : <https://debuggercafe.com/pytorch-imagefolder-for-training-cnn-models/>

Created with  on Weights & Biases.

http://localhost:8385/lizzafomenko/aim_imagenet_classification/reports/Tiny-ImageNet-200-classification-using-CNN-models--VmIldzoyOQ